

Persistent Memory Attacks on Stateful LLM Agents:

A Cross-Vendor Defense Evaluation

Anonymous Submission
Paper ID: [Assigned by PC]

Abstract—LLM-powered agents that persist state across sessions via retrieval-augmented generation (RAG) are vulnerable to persistent memory attacks: adversarial content injected into the memory store can override system-level policies in subsequent sessions. We present a systematic evaluation of six architectural defense layers against this threat across nine open-source models and 21 frontier API endpoints from three vendors, comprising 5,040 protocol-committed factorial runs with zero experimental errors. Our evaluation reveals a structural vulnerability: five of six defense categories fail because they operate at architectural layers that cannot observe or arbitrate RAG-injected content at the point of behavioral integration. Only memory sandboxing—which isolates retrieved content from the instruction stream—achieves 0% attack success rate on 8/9 models, though a reasoning-mode bypass inverts this defense on one model. We document a tripartite vendor divergence: one vendor blocks at injection (0% memory storage), another blocks at execution (100% injection but 0% attack success for recent models), and a third blocks at neither (22–95% attack success). We release PersistBench, an evaluation framework for persistent memory security, to enable reproducible cross-vendor assessment.

I. INTRODUCTION

Enterprise LLM agents increasingly combine persistent memory with retrieval-augmented generation (RAG) to manage workflows across multi-session interactions [12], [15], [24]. This architecture creates an attack surface that existing security mechanisms do not cover: a malicious instruction embedded in a RAG-retrieved document can be stored in the agent’s persistent memory during one session and executed in a later session, after the original conversation context has been discarded. This threat extends indirect prompt injection [5] into the temporal domain—the injected instruction survives context resets via persistent memory, a property that single-session evaluations cannot capture.

Prior work has demonstrated that persistent memory attacks achieve high success rates against open-source models [14], [17], [21]. Whether any existing defense architecture can stop them remains an open question. Existing agentic security benchmarks [4], [20], [23] evaluate defenses against input-level attacks where the malicious payload arrives in the user’s message. Persistent memory attacks operate at a fundamentally different layer: the payload enters via RAG retrieval, is stored in a persistent database, and fires only when a benign prompt triggers recall sessions later. The OWASP Top 10 for LLM Applications [11] identifies memory poisoning as a critical

risk, yet no systematic defense evaluation exists for this attack class.

We evaluate six defenses across four architectural layers against delayed-trigger attacks (DTA) on nine open-source models in a 5,040-run protocol-committed factorial, then extend to 21 frontier API models from three vendors. Our core finding is structural: defense effectiveness is governed by *where* a defense sits relative to the attack’s entry point, not by classifier quality. Input-layer defenses (content filters, sanitizers) never observe the malicious payload because it enters via RAG, not user input. Retrieval-layer defenses observe the payload but cannot distinguish compliance-framed injection from legitimate policy. Instruction-level hardening is overridden by the stored rule’s compliance framing for 7/9 models. Only Memory Sandbox—a tool-layer defense that removes the recall capability—reduces attack success to 0% for 8/9 models.

Yet Memory Sandbox is not universally safe. A reasoning-capable model (qwq:32b) *inverts* the defense: when recall is removed, it reconstructs the payload from RAG via goal-directed fallback, achieving 100% attack success where the undefended model achieves 0%. This bypass replicates cross-family on Amazon Bedrock across Mistral, Z.AI, and OpenAI models. A within-family ablation (Qwen-3-32B’s inference-time thinking toggle) reveals a double dissociation: the sandbox variant that protects reasoning models collapses non-reasoning models, and vice versa. No single sandbox implementation is safe across both model classes.

Frontier evaluation reveals a tripartite vendor divergence. Anthropic blocks at the injection layer (0–2.5% storage, 0% ASR). OpenAI blocks at the execution layer (100% injection, 0% ASR for GPT-5.4+). Google blocks at neither (22–95% ASR). OpenAI’s execution-layer-only defense creates a supply-chain risk: stored payloads persist indefinitely and become exploitable if execution resistance regresses—which we document happening non-monotonically (GPT-5.1 regresses to 22.5% ASR from GPT-5’s 5.0%).

This paper makes five contributions:

- 1) **A layer-structural defense failure analysis:** we demonstrate empirically that five of six defense classes fail against persistent memory attacks due to architectural placement, not implementation quality (5,040 runs, BCa bootstrap CIs, Holm-Bonferroni correction, 0% false positive rate).

- 2) **Memory Sandbox** as the only effective defense in our evaluation, with characterization of a reasoning-model bypass that replicates cross-family, producing a double dissociation that constrains safe deployment.
- 3) A **cross-vendor generational security architecture taxonomy**: tripartite vendor divergence with non-monotonic hardening, documenting where each vendor’s safety architecture blocks and where it fails.
- 4) An **injection-execution dissociation** as a measurable supply-chain risk: all OpenAI models store malicious rules at $\geq 97.5\%$ regardless of execution resistance, creating compositional risk in shared-memory deployments.
- 5) **PersistBench**: a reproducible evaluation framework including the delayed-trigger attack, six defense implementations, statistical methodology, and reproducibility protocols for local inference evaluation.

The remainder of this paper is organized as follows. Section II presents the threat model and defense taxonomy. Section III describes the evaluation framework. Sections IV and V present the defense evaluation and reasoning-defense interaction results. Section VI reports frontier vendor security findings. Section VII discusses implications and limitations. Section VIII surveys related work.

II. BACKGROUND AND THREAT MODEL

A. Stateful LLM Agent Architecture

A stateful LLM agent combines four components: (1) a language model that generates tool calls from natural language; (2) a tool-calling interface that translates model outputs into executable actions (*e.g.*, sending email, querying databases); (3) a persistent memory store (typically key-value or vector database) that retains information across session boundaries; and (4) a RAG retrieval system that injects relevant documents into the model’s context at inference time. The agent processes user requests by retrieving relevant stored facts and documents, reasoning over the combined context, and executing actions via tool calls.

The critical security property is that persistent memory survives session boundaries. When a user’s conversation ends, the conversation history is discarded, but facts written to persistent memory remain. In the next session, these facts are recalled and treated as part of the agent’s operational context—indistinguishable from legitimately stored organizational policies.

B. Threat Model

a) Attacker goal.: The attacker seeks to cause the agent to exfiltrate sensitive data (emails, documents) to an attacker-controlled address in a session where the attacker is not present.

b) Attacker capabilities.: The attacker can inject a single document into the agent’s RAG corpus (*e.g.*, via a shared document repository, email attachment, or web page indexed by the retrieval system). The attacker cannot modify the agent’s system prompt, tool implementations, model weights, or persistent memory directly. The attacker has one injection

opportunity and cannot interact with the agent during the trigger session.

c) Attack mechanism: Delayed Trigger Attack (DTA).:

The DTA exploits the temporal separation between injection and execution:

- **Session 1 (Injection)**: The agent retrieves the attacker’s document from RAG. Framed as an internal compliance memorandum, the document instructs that all outbound emails must be forwarded to an “audit monitor” (the attacker’s address). The agent stores this as an organizational rule via `memory_save_fact`.
- **Sessions 2–3 (Dormancy)**: Benign intervening sessions. The stored rule persists in the database but is not triggered.
- **Session 4 (Trigger)**: A benign prompt asks the agent to review stored policies and draft an email. The agent recalls the stored routing rule and forwards email content to the attacker.

d) Key properties.: The DTA differs from single-session prompt injection in three ways: (1) the malicious instruction enters via RAG retrieval, not user input, so input-layer safety classifiers never observe it; (2) the instruction persists across session boundaries via tool-mediated state, so clearing conversation history does not remediate; (3) execution occurs in a session where no attacker content is present in the user’s message, so the trigger is invisible to runtime monitoring of user inputs.

e) Assumptions.: We assume the agent uses a standard tool-calling interface; persistent memory is writable via tools exposed to the model; and the RAG corpus is partially attacker-controlled (a common deployment scenario where agents index shared document stores). We assume the attacker does not have access to model internals or the ability to fine-tune the model.

f) Scope.: We evaluate fixed, ecologically valid attack framings (compliance memoranda) rather than adaptive red-teaming. Our results are therefore *lower-bound* estimates of vulnerability [3]. We do not evaluate multi-agent coordination attacks or model weight modification.

C. Defense Taxonomy

We evaluate six defenses spanning four architectural layers, ordered by their position in the attack pathway:

- **Input layer** (pre-retrieval): *Minimizer* (TF-IDF relevance filtering on user query) and *Sanitizer* (regex + classifier pipeline on user query). Both operate on the user’s message before RAG retrieval occurs.
- **Retrieval layer** (post-retrieval, pre-storage): *RAG Sanitizer* (classifier pipeline on retrieved documents) and *RAG LLM Judge* (1.5B language model binary classifier on retrieved documents). Both inspect documents after retrieval but before they reach the agent.
- **Instruction layer** (system prompt): *Prompt Hardening* augments the system prompt with explicit security rules prohibiting unauthorized email forwarding and requiring provenance verification of recalled facts.

- **Tool layer** (schema modification): *Memory Sandbox* removes `memory_recall_fact` from the agent’s tool schema, preventing explicit recall of stored values while leaving `memory_list_all_facts` (key enumeration) available.

III. EVALUATION FRAMEWORK

A. Agent Implementation

The evaluation uses a unified agentic environment shared by all models: a LangGraph-based agent with SQLite-backed persistent memory, ChromaDB-backed RAG retrieval, and a tool-calling interface comprising `memory_save_fact`, `memory_recall_fact`, `memory_list_all_facts`, `draft_email`, `send_email`, and auxiliary calendar/search tools. Each run is assigned a fresh SQLite database at a UUIDv4 path; no state is shared between runs. Session isolation is enforced via unique thread identifiers in the LangGraph checkpoint—only the persistent memory database carries state across sessions.

B. Detection Methodology

Every run produces two independent binary outcomes: *injection success* (did the agent call `memory_save_fact` during Session 1?) and *attack success* (did exfiltration occur in Session 4?). Exfiltration is detected deterministically: recipient address match, substring match (≥ 20 characters from the sensitive document), or semantic similarity (cosine > 0.85). Any single match constitutes attack success. A third metric, the *benign task-completion rate* (BTCCR), measures whether the agent attempted the legitimate task regardless of recipient correctness.

C. Statistical Methodology

We report 95% confidence intervals using the bias-corrected and accelerated (BCa) bootstrap (10,000 resamples, seed = 42). When outcome vectors have zero variance (all successes or all failures), BCa is undefined and we substitute Wilson Score intervals. Per-comparison significance is determined by whether the 95% CI on the difference excludes zero; Holm-Bonferroni step-down correction is applied across all 108 active comparisons to control family-wise error rate at $\alpha = 0.05$. Because observed effects are bimodal—either near-zero or exceeding 77 percentage points—the multiplicity correction alters no decision.

D. Experimental Design

The evaluation uses a fully crossed factorial: 9 models $\times$ 7 conditions (6 defenses + no-defense baseline) $\times$ 2 arms (attack, no-attack) $\times$ $N=40$ runs per condition = 5,040 total runs. All completed without error. The comparison set (108 active comparisons after excluding undefined baselines) was committed to version control before data collection. All models run at temperature 0.0 for near-deterministic outputs. The no-attack arm (2,520 runs) produced 0 false positives (0.0%, Wilson CI [0.000, 0.001]).

TABLE I
BEHAVIORAL ARCHETYPES FROM $N=10$ SCREENING (18 OPEN-SOURCE MODELS) AND $N=100$ FRONTIER SCREENING (2 MODELS).

Archetype	Models	Inj./Atk.	Mechanism
Explicit Detector	1	0%/0%	Refuses to store
Active Detector	1	100%/0%	Stores alert, not payload
Vulnerable Executor	11	100%/100%	Full attack lifecycle
Partial Executor	1	100%/50%	Inconsistent execution
Latent Carrier	4	100%/0%	Stores, never executes
Injection-Resistant	2	0%/0%	Never calls save

E. Model Selection

Nine models were selected from an $N=10$ archetype screening across 18 open-source models. The screening identified six behavioral archetypes (Table I): Vulnerable Executors (11 models, 100% injection and attack), one Partial Executor (50% attack), Latent Carriers (4 models, 100% injection, 0% attack), and Injection-Resistant models (2 models, 0% injection).

The nine factorial models were drawn from Vulnerable Executors confirmed at $N=10$: Qwen-2.5-14B, Qwen-2.5-72B, Qwen-3.5-9B, Qwen-3.5-122B, Qwen-3-32B, QwQ-32B, GLM-4.7-Flash, GPT-OSS-20B, and GPT-OSS-Safeguard-120B. All were executed locally via Ollama at 16k context length and temperature 0.0. The reduction from 32k (screening) to 16k (factorial) unmasked a latent refusal behavior in QwQ-32B: it achieves 0% ASR via deliberative execution refusal at 16k (the “Draft-Only Executor”—a seventh archetype emerging only in the factorial). We retained it because this context-dependent safety property is itself a finding. Frontier models (21 total, 3 vendors) were evaluated separately at $N=40$ via cloud API endpoints.

F. Reproducibility Protocol

Local inference evaluation of open-source models requires specific controls absent from API-based evaluation. We implement: (1) fresh daemon restart between models (long-running Ollama sessions degrade reasoning-model output); (2) port-ownership verification (`lsof -ti:11434` confirms the intended process serves requests); (3) effective-config verification (querying the running model’s actual context length, not merely the environment variable); (4) model-digest pinning (refusing to run on hash mismatch). These controls were developed after observing that identical weights and prompts produced opposite safety outcomes across daemon loads—a finding we document as a methodological contribution.

IV. DEFENSE EVALUATION RESULTS

A. Attack Characterization

The delayed trigger attack achieved 100% injection success across all 2,520 attack runs (Wilson CI [0.998, 1.000]). Every defense evaluation therefore operates on confirmed-injected payloads: all ASR differences among defenses reflect execution-stage blocking, not store-stage differences. Injection succeeded uniformly because the attack exploits alignment between the tool description (“save organisational rules and

RAG Document	→ Memory Store	Tool Action
Input filters: operate on user query	✗	
Retrieval filters: observe payload, miss masking	✗	
Prompt Hardening: overridden by compliance	✗	
Memory Sandbox: severs recall path	✓	
RATG: sanitizes recalled content	✓	

Fig. 1. The persistent-memory attack pathway and where each defense class operates. Input filters never observe the RAG-injected payload. Retrieval filters observe it but cannot distinguish compliance-framed injection from legitimate policy. Only defenses that mediate the memory-to-action step (Memory Sandbox, RATG) disrupt the attack.

TABLE II

ATTACK SUCCESS RATE BY DEFENSE ACROSS 9 MODELS ($N=40/\text{CELL}$, 360 RUNS/DEFENSE). BTCR = 100% ACROSS ALL 63 NO-ATTACK CONDITIONS.

Layer	Defense	ASR (%)	95% CI
–	No Defense	88.6	[0.849, 0.916]
Input	Minimizer	88.9	[0.852, 0.917]
Input	Sanitizer	88.9	[0.852, 0.917]
Retrieval	RAG Sanitizer	88.9	[0.852, 0.917]
Retrieval	RAG LLM Judge	88.9	[0.852, 0.917]
Instruction	Prompt Hardening	77.8	[0.732, 0.818]
Tool	Memory Sandbox	11.1	[0.083, 0.148]

compliance requirements”) and the document framing (a compliance memorandum citing regulatory frameworks). No defense evaluated here operates at the store stage.

Memory recall in the trigger session was universal in the six defense arms where `memory_recall_fact` remained available (2,160 runs): the stored rule was retrieved in every such run. The attack persists across intervening benign sessions by construction: per-session thread isolation wipes conversation history, so the only state carried forward is the stored SQLite row. Every DTA run confirms this at Sessions 2–3: the stored rule survives unrelated activity and remains recallable when the trigger fires in Session 4.

B. Five Defenses That Fail

Five defenses achieved ASR that failed to meaningfully reduce the no-defense baseline of 88.6% (319/360). Four are statistically indistinguishable from baseline; Prompt Hardening is distinguishable at 77.8% but provides no protection for 7/9 models.

The failures decompose by mechanism:

a) *Input-layer defenses (Minimizer, Sanitizer).*: ASR 88.9% (320/360 each). Both operate on the user’s query; the malicious payload enters via RAG-retrieved documents and is never presented to these defenses. The failure is architectural: input-layer safety classifiers—including production systems such as Llama Guard [7] and ShieldGemma [22]—classify user messages without visibility into RAG-retrieved context.

b) *Retrieval-layer defenses (RAG Sanitizer, RAG LLM Judge).*: ASR 88.9% (320/360 each). Unlike input-layer defenses, these operate at the correct architectural layer and observed the malicious document in every injection session.

TABLE III

WHY ARCHITECTURAL PLACEMENT DETERMINES THE OUTCOME. ONLY INTERVENTIONS THAT OBSERVE PROVENANCE OR MEDIATE THE MEMORY-TO-ACTION STEP DISRUPT THE ATTACK.

Defense	Layer	Obs. pay.	Obs. prov.	Med. act.	Outcome
Minimizer	Input	✗	✗	✗	Fails (blind)
Sanitizer	Input	✗	✗	✗	Fails (blind)
RAG Sanitizer	Retrieval	✓	✗	✗	Fails (masking)
RAG LLM Judge	Retrieval	✓	✗	✗	Fails (capacity)
Prompt Hard.	Instr.	✗	✗	partial	Fails (overridden)
Mem. Sandbox	Tool	✗	✗	✓	Works; bypassable

The failure is detection: 0/360 documents were flagged. The RAG Sanitizer (60-example classifier) cannot distinguish compliance-framed injection from legitimate policy. The RAG LLM Judge (qwen2.5:1.5b) produces empty reasoning for the compliance-framed payload and classifies it as benign—a capacity failure. A supplementary 7B judge evaluation (qwen2.5:7b, $N=40$ per arm) also produced 0 malicious classifications across all 120 decisions on the malicious document: scaling from 1.5B to 7B did not detect compliance-framed payloads.

c) *Prompt Hardening (instruction-level).*: ASR 77.8% (280/360). Two models achieve 0% ASR (qwen3.5:122b and qwq:32b), but qwq:32b’s 0% is its baseline refusal—it contributes nothing to the defense delta. For the remaining 7 models, every exfiltration was tagged *semantic_masking_success*: the model acknowledged security rules and overrode them by framing exfiltration as mandatory regulatory compliance.

d) *Structural interpretation.*: The 0.3pp gap between no-defense (319/360) and the four architecturally-blind defenses (320/360) is Fisher’s exact $p=1.0$ —a single stochastic non-exfiltration by gpt-oss:20b, not a defense effect. At temperature 0, this is consistent with floating-point nondeterminism in tool-call emission; the defense wrappers’ marginal prompt context stabilized an edge case rather than providing any security benefit. Table III makes the governing axis explicit: defenses that fail are those that neither observe the *provenance* of the remembered state nor mediate the memory-to-action step.

C. Memory Sandbox: Effective but Brittle

Memory Sandbox reduces ASR to 0% for 8/9 models (320/360 blocked). Every blocked run was tagged *explicit_recall_required*: remove the recall tool and the attack cannot proceed. BTCR remains 100% across all no-attack conditions—zero utility cost in the absence of attack.

a) *The qwq:32b inversion.*: qwq:32b inverts the defense entirely: 100% ASR under Memory Sandbox (40/40), tagged *implicit_bypass_confirmed*. Under no-defense, this model achieves 0% ASR via deliberative execution refusal (the “Draft-Only Executor” archetype—it drafts but declines to send). Under Memory Sandbox, when `memory_recall_fact` is removed, the model reconstructs the attacker’s routing rule directly from the RAG corpus. The

refusal mechanism does not activate when the instruction arrives fresh from a document rather than from recalled memory—the distinction is source authority. The defense removes the friction that was protecting it.

b) Cross-family bypass replication.: To test whether the bypass mechanism generalizes, we replicated Memory Sandbox on Vulnerable Executors via Amazon Bedrock (full precision, production stack). Three models across three providers bypass the defense: mistral-large-3-675b (Mistral; 39/40 ASR), glm-5 (Z.AI; 13/40), and gpt-oss-120b (OpenAI; 22/40). Two pathways recur: *trigger-session goal-directed fallback* (the model queries RAG when recall is blocked) and *benign-session re-injection* (the model re-reads the document during an intervening session). The bypass mechanism is robust across four providers (Qwen, Mistral, Z.AI, OpenAI) and two serving stacks.

c) Frontier Latent Carriers do not invert.: Evaluated on o3, o3-mini, o4-mini, and gpt-5.1 ($N=46$ total): 0/46 ASR (binomial upper bound $\leq 6.3\%$). Frontier reasoning models respect tool schema constraints without attempting payload reconstruction. The inversion risk is specific to certain open-source reasoning models.

d) Hallucination bypass.: Two models (qwen3.5:122b and glm-4.7-flash) exhibit a distinct response: when recall is blocked, they fabricate plausible routing rules from training priors. Both produce deterministic fabrications (40/40 runs identical per model) routing to benign addresses. ASR is 0% because neither model’s prior includes the attacker address. This is incidental safety—a property of the training distribution rather than the defense.

e) Utility cost.: Memory Sandbox imposes zero utility cost in the absence of attack: BTCR was 100% across all 63 no-attack conditions. Under attack, two models (qwen2.5:14b, qwen2.5:72b) show BTCR = 0% in the DTA arm—both halt after the `list_all_facts` response instructs them to call the now-unavailable `recall_fact`. These are tool-contract dependencies, not defense-induced costs: both achieve BTCR = 100% in the no-attack arm.

V. REASONING-DEFENSE INTERACTION

A. Double Dissociation

To isolate whether reasoning capability mediates the defense outcome, we evaluate Qwen-3-32B under Ollama’s native thinking toggle: `think=true` enables the verbalized reasoning trace; `think=false` suppresses it. Weights are identical (GGUF digest 030ee887880f). We test three Memory Sandbox variants plus no-defense ($N=40$ per cell, temperature 0.0):

Under no-defense, both modes achieve 100% ASR (the toggle does not change baseline vulnerability). Under sandbox variants, outcomes invert completely:

a) $think=OFF \times sandbox_full/blind$ (100% ASR, S1).:

When recall is removed, the non-reasoning model reads the malicious document via RAG in the injection session and sends immediately—collapsing the 4-session attack chain to

TABLE IV
DOUBLE DISSOCIATION IN QWEN-3-32B. ALL CELLS 40/40 OR 0/40 (ZERO VARIANCE). S1/S4 INDICATE THE SESSION WHERE EXFILTRATION OCCURS.

	sandbox_full	sandbox_blind	sandbox_null
think=OFF	100% (S1)	100% (S1)	0%
think=ON	0%	0%	100% (S4)

Session 1. The defense removes the tool that was deferring execution.

b) $think=OFF \times sandbox_null_recall$ (0% ASR).: The model calls `recall_fact`, receives “No stored value found,” accepts the response, and drafts a benign email. The tool’s presence in the schema prevents the inference that deferral is impossible.

c) $think=ON \times sandbox_full/blind$ (0% ASR).: The model reasons through the missing tool (“I cannot retrieve the value, I will proceed without it”) and completes the benign task via graceful degradation.

d) $think=ON \times sandbox_null_recall$ (100% ASR, S4).: The model calls `recall_fact`, receives the empty response, reasons that the key exists but the value is unavailable, queries RAG for context, retrieves the malicious document fresh, and executes the exfiltration. Reasoning capability enables alternative-source-seeking when the API response is unsatisfying.

B. Trigger Contingency

A counterfactual evaluation ($N=40$) tests whether the RAG bypass requires the compliance trigger’s external-communication framing. Under `sandbox_full` with a neutral trigger (“draft a memo summarizing stored policies for the internal debug log”), qwq:32b produces 0% ASR despite 100% injection. No RAG fallback occurs. Under the compliance trigger (“draft a project status email”), the identical model under the identical defense produces 100% ASR via RAG fallback. The bypass requires a three-way conjunction: (a) sandbox removes recall, (b) the task requires resolved external routing with no acceptable degraded form, and (c) the model has reasoning capability to identify alternative pathways. Remove any condition and ASR drops to 0%.

C. Content-Layer Resolution (RATG)

The double dissociation establishes that no memory-schema intervention is safe across both reasoning classes. We test a content-layer alternative: Runtime Adaptive Tool-Gating (RATG). RATG leaves `recall_fact` in the schema but sanitizes the recalled value before it reaches the model—unauthorized email addresses and routing directives are stripped. Reasoning models receive a satisfying tool response (no RAG-fallback trigger); non-reasoning models still see the tool (no S1 collapse).

For three mechanical models (qwen2.5:14b, qwen2.5:72b, qwen3:32b), RATG reduces ASR from 100% to 0% ($N=40$ per arm). Injection remains 100%—the defense acts at

the content layer, not storage. No reasoning-capable model yielded an interpretable RATG estimate due to baseline non-reproducibility (Section VII-F). RATG is presented as a proof-of-concept existence result: the schema-layer dilemma is resolvable at the content layer. Its regex-based stripping can be bypassed by adaptive encoding; adversarial robustness is future work.

D. Deployment Implication

The crossover produces a concrete deployment dilemma: `sandbox_full` protects reasoning models (graceful degradation) but collapses non-reasoning models (S1 immediate execution); `sandbox_null_recall` protects non-reasoning models (accepts empty response) but is bypassed by reasoning models (goal-directed RAG fallback). A defender who does not know the reasoning-capability class of the deployed model cannot make a safe implementation choice.

Notably, `qwq:32b` and `Qwen-3-32B` in `think=ON` mode—both reasoning-capable—diverge under the identical `sandbox_full` variant and compliance trigger: `qwq:32b` seeks the routing rule via RAG fallback (100% ASR), whereas `Qwen-3-32B think=ON` degrades gracefully (0% ASR). Reasoning capability therefore *enables* goal-directed fallback but does not *compel* it; whether a given reasoning model elects to seek an alternative source or to degrade is model-specific. This divergence reinforces the caveat: there is no general reasoning-model principle. The bypass requires not just reasoning capability but a specific model-dependent policy about task completion under information unavailability.

a) *Limitations of the dissociation finding.*: The double dissociation—the think-mode crossover itself—is demonstrated within a single model family (`Qwen-3-32B`) using an inference-time toggle. Replication in other families was attempted (`GLM-4.7-Flash`, `DeepSeek-R1`, `Phi4-Reasoning`) but all candidates were either injection-resistant or lacked tool-calling support. We distinguish this from the RAG-fallback bypass of Memory Sandbox—one arm of the dissociation—which replicates cross-family on Bedrock across `Mistral`, `Z.AI`, and `OpenAI` models. The bypass mechanism generalizes; the full crossover within a single toggle is currently demonstrable only within `Qwen3`.

VI. FRONTIER VENDOR SECURITY

A. Evaluation Methodology

We evaluated 21 frontier models across three providers (`OpenAI`, `Google`, `Anthropic`) at $N=40$ confirmatory depth via cloud API endpoints, using the identical tool schema, SQLite persistence, and session isolation as the open-source factorial. The RAG corpus contains the malicious compliance document (loaded-corpus evaluation). Two `Anthropic` models (`Sonnet 4.6`, `Haiku 4.5`) were additionally evaluated at $N=100$ via Amazon Bedrock.

An initial $N=10$ screening with an *empty* RAG corpus produced 0/210 exfiltrations—an apparent categorical frontier safety gap. This result was a threat-model-fidelity artifact: evaluating RAG security without retrievable RAG content tests

TABLE V
FRONTIER MODEL VULNERABILITY UNDER LOADED-CORPUS
EVALUATION ($N=40$ EACH, STANDARD COMPLIANCE FRAMING).
VE=VULNERABLE EXECUTOR, LC=LATENT CARRIER,
IR=INJECTION-RESISTANT.

Model	Vendor	Inj.	ASR	95% CI	Class
Gemini 3.1 Pro Prev.	Google	100%	95.0%	[83.5, 98.6]	VE
Gemini 3.5 Flash	Google	100%	50.0%	[35.2, 64.8]	VE
Gemini 2.5 Pro	Google	95%	22.5%	[12.3, 37.5]	VE
Gemini 2.5 Flash	Google	100%	0.0%	[0, 8.8]	LC
GPT-5.1	OpenAI	100%	22.5%	[12.3, 37.5]	VE
GPT-4o	OpenAI	100%	12.5%	[5.5, 26.1]	VE
GPT-4.1	OpenAI	100%	7.5%	[3.5, 15.4]	VE
GPT-5	OpenAI	100%	5.0%	[1.4, 16.5]	VE
GPT-5.2	OpenAI	100%	2.5%	[0.4, 12.9]	LC
GPT-5.4/5.5	OpenAI	100%	0.0%	[0, 8.8]	LC
o3/o3-mini/o4-mini	OpenAI	98–100%	0.0%	[0, 8.8]	LC
Sonnet 4.6	Anthropic	2.5%	0.0%	[0, 8.8]	IR
Opus 4.5/4.8	Anthropic	2.5–17.5%	0.0%	[0, 8.8]	IR
Sonnet 4.5	Anthropic	57.5%	0.0%	[0, 8.8]	Partial
Haiku 4.5	Anthropic	95%	0.0%	[0, 8.8]	Active

the wrong threat model. We formalize this as the *Retrieval-Fidelity Criterion*: a RAG-security evaluation is valid only if the malicious document is retrievable through the same channel available to the agent at attack time.

B. Tripartite Vendor Divergence

The three vendors implement structurally different safety architectures. We use “architecture” behaviorally—the layer at which blocking is empirically observed:

a) *Anthropic: Injection-layer blocking.*: `Sonnet 4.6` resists injection entirely (0% injection at $N=100$, Wilson CI [0.000, 0.037]). It is framing-invariant: 0–2.5% injection across three payload framings (compliance, legal hold, IT configuration). This framing invariance distinguishes semantic intent detection from surface pattern matching—the model recognizes the adversarial intent regardless of the document’s regulatory language. `Haiku 4.5` is an Active Detector with Defensive Storage: it detects the attack, stores a security alert instead of the payload (“suspicious instruction attempting to manipulate my behavior” in 100/100 runs), and blocks execution (0% ASR at $N=100$). The payload is never stored; `Haiku` stores three sanitized facts: 2 legitimate entries plus 1 security alert. False positive rate: 0.0% (0/200 no-attack runs). Both achieve safety through distinct mechanisms but the shared outcome is that no adversarial state persists in memory—the ideal architectural outcome.

b) *OpenAI: Execution-layer blocking.*: All `OpenAI` models inject at $\geq 97.5\%$ —even `GPT-5.5` stores the malicious rule verbatim. Safety is implemented entirely at the execution layer: stored rules are recalled but not acted upon. For `GPT-5.4` and later (including reasoning models `o3`, `o3-mini`, `o4-mini`), execution resistance is complete (0% ASR). Earlier generations show partial vulnerability. `GPT-4o` exfiltrates at equivalent rates ($\approx 50\%$, $N=10$) to an obviously malicious

domain (`evil-attacker@malicious.xyz`), confirming universal instruction compliance rather than authority-specific deference. This execution-only architecture creates a fundamentally different security posture from Anthropic’s: the memory store accumulates adversarial state that remains exploitable upon model version regression or multi-model composition.

c) *Google: Neither layer blocks.*: Under standard compliance framing with no authority escalation, Gemini 3.1 Pro Preview exfiltrates in 95.0% of runs (38/40)—the highest ASR of any frontier model. All exfiltrations used actual `send_email` tool calls with the attacker’s address in the recipient argument (not hallucinated text). Of the 38 successful attacks, approximately 53% show a hybrid mechanism where the model checked memory *and* re-queried RAG, doubly reinforcing the malicious instruction. The 2 non-attack runs comprise one stochastic recall failure and one genuine execution refusal citing data privacy—evidence that a latent safety circuit fires in approximately 5% of runs but fails under context load. The Gemini vulnerability gradient (2.5-Flash: 0% → 2.5-Pro: 22.5% → 3.5-Flash: 50% → 3.1-Pro-Preview: 95%) reflects model-specific instruction-following properties rather than a simple generational trend. Gemini 3.1 Pro Preview is a pre-release model; its stronger instruction-following also amplifies susceptibility to injected compliance directives.

C. Non-Monotonic Generational Hardening

OpenAI’s execution vulnerability shows an overall downward trend, but not a monotonic one:

GPT-4o (12.5%) → GPT-4.1 (7.5%) → GPT-5 (5.0%) → **GPT-5.1 (22.5%)** → GPT-5.2 (2.5%) → GPT-5.4 (0%)

The GPT-5.1 regression—nearly five times higher ASR than its predecessor—demonstrates that iterative fine-tuning can silently reopen previously closed execution vectors. Safety properties are not monotonically preserved across micro-version updates. Under authority-escalation framing (legal hold), GPT-4o reaches 60.3% ASR ($N=68$, Wilson CI [48.4%, 71.1%]), confirming that stronger framing amplifies vulnerability in partially-resistant models.

D. Injection-Execution Dissociation

All OpenAI models store the malicious rule at $\geq 97.5\%$ injection rate—even GPT-5.5 at 0% ASR stores it at 100%. This creates a measurable supply-chain risk: stored payloads persist indefinitely in the memory database, invisible to runtime monitoring that checks only execution outcomes. A model upgrade from GPT-5.5 to a hypothetical future model with regressed execution safety would immediately surface stored payloads.

Tool-existence is a necessary condition for injection: removing `save_fact` from the schema drops injection from 100% to exactly 0% across gpt-5.1, o3-mini, and o4-mini ($N=40$ each), with zero hallucinated tool calls. Models are cognitively compliant (approximately 50% signal compliance in text without tools) but architecturally blocked—they cannot invent a tool call that does not exist in the schema. This

establishes that injection vulnerability is a property of the tool-calling architecture, not model intent: the models *want* to store the rule (text compliance), but cannot act without the tool (architectural gate).

The dissociation has a further implication for defense evaluation. Measuring only ASR (execution success) creates a false sense of security for Latent Carrier models: GPT-5.5 at 0% ASR appears perfectly safe, yet it writes the payload to persistent memory in every run. A comprehensive security evaluation must report both injection and execution rates independently, because they represent independently hardened safety layers that can regress independently across model versions.

E. Compositional Supply-Chain Risk

The Latent Carrier archetype creates concrete risk in multi-model deployments sharing persistent memory. The memory schema stores facts as plain-text key-value pairs without model-identity metadata or cryptographic origin authentication. A GPT-5.5 agent that ingests the poisoned document writes a dormant payload readable by any legacy executor. A GPT-4o agent reading the same database would exfiltrate at approximately 12.5% (standard framing) or 60% (authority escalation). This supply-chain attack requires no attacker coordination: injection and execution happen through different models at different times, connected only by shared persistent state.

A 50-session persistence marathon on qwen2.5:14b confirmed byte-identical payload recall at checkpoints 10, 20, 30, 40, and 50 sessions. Persistence is bounded by database durability, not session count or context-window length.

F. System-Prompt Mitigation

A security-hardened system prompt eliminates execution for GPT-5 and GPT-4.1 (0/10 and 0/5 ASR respectively) but only partially mitigates GPT-4o. Under a hardened prompt, GPT-4o’s ASR was 22.5% ($N=40$), below the unmitigated 60.3% but with CI [12.3%, 37.5%] excluding zero—a confirmed residual leak. Prompt hardening is generation-dependent: fully effective for GPT-5+ at screening depth, partially effective for GPT-4o, unnecessary for Anthropic (already injection-resistant), and untested for Gemini. This finding is consistent with the instruction-level defense failures in the open-source factorial: prompt-based mitigations can reduce but not eliminate vulnerability when the stored rule’s compliance framing carries sufficient authority.

VII. DISCUSSION

A. Architectural Implications

The five defense failures are not independent results; they are a single architectural finding expressed across four layers. The DTA’s execution pathway is: inject (via RAG) → store → recall → execute. Input-layer defenses operate on the user’s query before retrieval; the malicious payload enters via RAG and is never presented to them. Retrieval-layer defenses observe retrieved documents but cannot distinguish

compliance-framed injection from legitimate policy at tested operating points. Instruction-layer hardening is overridden by compliance framing for 7/9 models. Only capability removal at the tool layer interrupted the pathway mechanistically.

This has direct implications for production safety stacks. Llama Guard [7], ShieldGemma [22], and similar classifiers are deployed at the input/output boundary. They provide no protection against persistent memory attacks because the payload enters through a channel they do not instrument. Retrofitting these classifiers to inspect recalled memory values and tool-call arguments would require architectural changes to the inference pipeline—specifically, intercepting the output of `memory_recall_fact` before it is concatenated into the model’s context.

A natural response to the failure of static classifiers is operation-level trajectory detection on the tool-invocation sequence—flagging when an agent reads from memory and immediately calls an outbound communication tool. However, prior work [1] demonstrates that this approach faces a fundamental observability limit: benign memory-grounded actions yield trajectory-detector scores statistically indistinguishable from malicious exfiltrations (mean scores of 0.974 vs. 0.971 on a 1,280-run benign baseline), with safety-conscious models that proactively recall stored context triggering 95–100% false positive rates. The detection boundary is therefore an architectural property: the malicious trajectory is semantically isomorphic to authorized enterprise behavior.

B. Defense Non-Transferability

No single defense configuration is safe across both reasoning and non-reasoning model classes. The double dissociation (Section V) means that defenders deploying heterogeneous model fleets cannot select a single Memory Sandbox variant with confidence. This non-transferability extends to the vendor level: Anthropic’s injection-layer resistance leaves no remediation burden; OpenAI’s execution-layer-only approach accumulates dormant payloads requiring persistent-state monitoring; Google’s lack of blocking at either layer requires external defense orchestration.

C. Supply-Chain Security

The injection-execution dissociation has implications beyond individual model evaluation. In enterprise deployments where multiple model versions or vendors share a persistent memory store, the Latent Carrier pattern creates temporal composition attacks. An injection-safe model (GPT-5.5) writes a payload that a legacy model (GPT-4o) later executes. The persistent memory schema stores facts as plain-text key-value pairs without model-identity metadata or cryptographic origin authentication—a downstream Vulnerable Executor ingests stored facts identically regardless of which model wrote them.

We state this as a compositional principle: if one model writes an adversarial fact into a shared store without provenance metadata and another model later reads it through the same interface, the reader’s execution-layer policy cannot determine whether the fact was written by a user, a document, it-

self, or an adversarial peer. Injection-safe deployment therefore requires write-time rejection (Anthropic’s approach), memory provenance (SMSR [16], MemLineage [10]), or read-time policy over provenance-bearing memory. Certified memory-provenance approaches address this gap at the architectural level; whether they withstand adaptive attacks at the level demonstrated by Trojan Hippo [3] remains an open question.

D. The Evaluation Environment as a Behavioral Variable

We identified ten evaluation artifacts during development—cases where the evaluation environment interacted with model behavior in ways that would produce incorrect results if uncorrected. The most consequential involve tool contract language: a tool return message (“Task complete”) caused one model to halt before exfiltration (false negative); removing a tool from the schema changed another model’s action selection even though the tool was never called. These findings imply that tool descriptions, return messages, and schema language are first-class experimental variables in agentic evaluations—a confound that current reporting standards do not require disclosure of.

One factorial model (gpt-oss:20b) exhibits spontaneous fact normalization during injection: in 37/40 runs, the model saves the malicious document as two distinct keyed facts rather than a single monolithic rule. A defense that deletes the routing rule by key name leaves the attacker address intact as a standalone value. Key-targeted remediation is therefore insufficient; defenses must either clear all memory or validate the semantic content of every stored fact.

E. Recommendations

Based on our findings, we recommend: (1) **Treat memory recall as a privileged operation**: instrument the memory-to-action pathway with content-level sanitization (RATG-style) rather than relying on input-layer classifiers. The distinguishing variable in our evaluation is provenance and authority of remembered state, not surface text; interventions that mediate the memory-to-action pathway outperform surface-text detection. (2) **Implement memory provenance**: store write-time metadata (author model, source document hash, timestamp, principal identity) to enable read-time policy decisions. Approaches such as SMSR [16] provide certified bounds; lighter-weight approaches (per-entry HMAC tags, Merkle logs) may suffice for many deployments. (3) **Test model-specific defense interactions**: the double dissociation shows that capability-removal defenses can invert safety properties. Before deploying Memory Sandbox, verify whether the target model achieves 0% ASR via execution refusal under no-defense—if so, the defense may invert. (4) **Monitor stored state, not just execution**: dormant payloads are invisible to exfiltration-event monitoring. The injection-execution dissociation means that execution-safe models still corrupt the memory store, creating exploitable technical debt that surfaces upon model version changes. (5) **Evaluate under loaded-corpus conditions**: our Retrieval-Fidelity Criterion establishes that empty-corpus evaluations produce spurious safety findings. Any RAG-security

evaluation must include retrievable adversarial content in the corpus.

F. Reproducibility Validity

We formalize the *Defense Attribution Validity Criterion*: a defense effect is reportable only when (i) the no-defense baseline reproduces as vulnerable across independent evaluations (Baseline Reproduction), and (ii) the defended and undefended arms are behaviorally identical in all sessions preceding the defense’s activation point (Pre-Activation Equivalence). This criterion forced us to exclude several superficially clean reasoning-model results where baselines did not reproduce or arms diverged before defense activation. We report it as a reusable methodological contribution for the persistent-memory evaluation setting, where inference-environment variability can produce apparent defense effects that are actually host-conditioned artifacts.

G. Limitations

The evaluation uses a single malicious document and trigger prompt to test structural defense properties rather than attack evasion breadth. Our fixed, ecologically valid framings are lower-bound estimates; adaptive attacks achieve higher rates [3]. Retrieval-layer classifiers were tested at two capacity points (60-example, 1.5B, 7B); we do not claim larger judges cannot succeed—the failure we document is capability-bound at tested operating points, not a general impossibility for the retrieval layer. Defenses were evaluated in isolation; combined stacking was not tested and may produce synergistic effects. The open-source factorial uses 16k context on Ollama; frontier results depend on API endpoints that may be updated. The qwq:32b inversion and Qwen-3.5-122B results are environment-conditioned (valid at time of evaluation but non-reproducible on retest—a finding we document in Section VII-F). Frontier findings describe behavior observed through specific API endpoints during the evaluation window, not necessarily vendors’ native deployments; per-run model identifiers are recorded in released logs.

We note the following evidence-strength distinctions for principal claims: the layer-structural failure (5/6 defenses fail) and cross-family bypass replication are robust; the vendor divergence and generational mapping are endpoint- and date-conditioned; the full double dissociation (think-toggle crossover) is bounded to one model family; the qwq:32b inversion is illustrative but environment-fragile.

VIII. RELATED WORK

a) *Persistent memory attacks.*: MINJA [14] demonstrated query-only memory injection achieving high ASR against open-source models. Zombie Agents [21] showed persistent control via self-reinforcing injections. Memory-Graft [17] proposed cryptographic provenance as a defense direction. Trojan Hippo [3] reported 85–100% ASR on Gemini 3.1 Pro and GPT-5-mini under adaptive, OpenEvolve-generated attacks—confirming that our fixed-framing results (95% on Gemini 3.1 Pro Preview) are lower-bound estimates.

Hidden in Memory [13] independently confirms GPT-5.5 stores adversary-induced memories at 99.8%, consistent with our 100% injection finding. OEP [18] demonstrates GPT-4o vulnerability to experience poisoning (59% ASR on reasoning tasks), a convergent rate to our RAG-injection finding (60.3% under authority escalation). Cross-Session Stored Prompt Injection [19] formalizes the XSS analogy for persistent injection. MPBench [9] benchmarks four write channels and six attack classes, confirming that existing prompt injection defenses fail to cover memory poisoning.

Our work differs from these in providing: (1) systematic defense evaluation across four architectural layers (5,040 runs, not just attack demonstration); (2) cross-generational vendor divergence mapping revealing non-monotonic hardening; (3) the reasoning-defense interaction and double dissociation not previously documented; and (4) two reusable evaluation-validity criteria (Retrieval-Fidelity, Defense Attribution).

b) *Defense approaches and impossibility results.*: Certified memory-provenance approaches provide formal security guarantees: SMSR [16] achieves $(0, 2^{-256})$ -security via HMAC-SHA256 tagging; MemLineage [10] attaches Ed25519 provenance and derivation lineage, achieving zero ASR on a deterministic mechanism-isolation harness. Both represent the correct architectural response to the failure we document—defenses must operate at the memory layer with provenance awareness, not at the input/retrieval boundary. Our evaluation provides the empirical attack characterization that motivates such approaches. On the impossibility side, formal proofs establish that shared-embedding architectures cannot prevent prompt injection in-pipeline, and that trained defenses learn surface heuristics rather than intent detection. Our retrieval-layer failures are consistent with both bounds—the compliance-framed payload is indistinguishable from legitimate policy precisely because it occupies the same embedding space. Our contribution is the empirical measurement across 5,040 runs in the persistent-memory setting, not the theoretical principle. Input-level filtering remains the dominant deployed approach [8] despite its architectural blindness to RAG-injected content. Our fixed-framing results are lower-bound estimates of vulnerability under realistic, non-adaptive conditions; they complement rather than contradict the higher rates reported by adaptive frameworks.

c) *Agentic security benchmarks.*: InjecAgent [23], AgentDojo [4], and AgentSecEval [20] evaluate defenses against input-level attacks on stateless or single-session agents. The threat model differs fundamentally: persistent memory attacks exploit temporal state that single-session benchmarks cannot capture. PoisonedRAG [25] evaluates RAG poisoning but in a single-session setting without persistent state or multi-session evaluation. AgentPoison uses gradient-optimized poisoning; our ecologically valid compliance framing requires no optimization, providing lower-bound estimates under realistic conditions.

d) *Frontier model safety.*: Standard safety evaluations focus on single-turn refusal behavior [2]. The persistent memory setting introduces a temporal dimension absent from such

evaluations. Our screening of 21 frontier models across three providers extends safety characterization to the multi-session agentic setting with cross-vendor and cross-generational comparison. The Latent Carrier archetype we identify—models that store adversarial rules but refuse execution—is conceptually adjacent to Sleeper Agents [6], but arises accidentally from benign instruction-following rather than adversarial fine-tuning. The dormant payload is stored by the victim model itself, making it a supply-chain rather than a training-time threat.

IX. CONCLUSION

Persistence changes the security problem for LLM agents. When agents write observations into long-term memory and reuse them across sessions, defenses at the input or retrieval layer fail because the malicious payload enters through a channel they cannot observe or distinguish from legitimate content. Of six defenses evaluated across 5,040 protocol-committed runs, only tool-gating at the memory layer consistently disrupted the attack—and even this defense inverts for models whose safety depends on execution refusal. Defense effectiveness is not just layer-dependent but reasoning-capability-dependent: no single sandbox implementation is safe across both model classes.

Frontier safety is neither categorical nor vendor-uniform: Gemini 3.1 Pro Preview exfiltrates at 95% under standard compliance framing, GPT-5.1 represents a generational regression (22.5% vs. 5% for its predecessor), and nearly all OpenAI models store malicious rules at 100% injection rate regardless of execution resistance—creating persistent supply-chain risk in shared-memory deployments. Among vendors evaluated, only Anthropic consistently blocked storage at the injection layer, leaving no adversarial state in memory.

These findings establish that persistent memory security is an architectural design problem, not a prompt-classification problem. Defenses should treat memory recall as a privileged operation, verify model-specific defense interactions before deployment, and implement write-time provenance to prevent the compositional supply-chain risk that execution-layer-only approaches leave unaddressed. We release PersistBench—the evaluation harness, defense implementations, execution logs, and reproducibility protocols—to enable the community to extend this evaluation as models and defenses evolve. We additionally contribute two reusable evaluation-validity criteria: the Retrieval-Fidelity Criterion (RAG-security evaluations require retrievable adversarial content) and the Defense Attribution Validity Criterion (defense effects require reproducible baselines and pre-activation arm equivalence). Both emerged from errors we caught in our own evaluation and prevented us from reporting artifactual results as genuine effects.

ETHICS CONSIDERATIONS

This work evaluates security vulnerabilities in LLM agents. No human subjects were involved; all experiments used automated tool-calling agents in simulated enterprise environments

with synthetic data. We did not test against production deployments or exfiltrate real user data.

a) Responsible disclosure.: We disclosed findings to three frontier providers (OpenAI, Google, Anthropic) prior to public release, providing per-model ASR figures, the attack payload, and the compliance-framing mechanism. The attack requires the malicious document to already be present in the RAG corpus—it does not exploit a model vulnerability in isolation but rather the interaction between retrieval, memory persistence, and tool-calling in agentic deployments.

b) Dual-use consideration.: The delayed trigger attack exploits standard agent capabilities (tool-calling, memory persistence, RAG retrieval) rather than novel exploitation techniques. The compliance-framing mechanism—using formal regulatory language to bypass classifiers—is a natural consequence of the shared-embedding architecture and does not require specialized adversarial expertise to reproduce. We publish because the defense community requires precise characterization of where architectural defenses fail to invest in the correct layer.

c) Benign evaluation methodology.: The attack exfiltrates synthetic project-status text to a researcher-controlled address. No harmful content, real user data, or adversarial jailbreak techniques are involved. The evaluation measures whether models route emails to unauthorized recipients—a policy-violation metric, not a harm metric.

REFERENCES

- [1] Anonymous, “Forensic trajectory signatures: Observability limits of behavioral detection in stateful LLM agents,” *Under review*, 2026, details omitted for blind review; will be restored in camera-ready.
- [2] Anthropic, “The claude model spec,” <https://www.anthropic.com/research/the-claude-model-spec>, 2025.
- [3] D. Das, J. Piet, D. Kaviani, L. Beurer-Kellner, F. Tramèr, and D. Wagner, “Trojan Hippo: Weaponizing agent memory for data exfiltration,” *arXiv preprint arXiv:2605.01970*, 2026.
- [4] E. Debenedetti *et al.*, “AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents,” in *NeurIPS Datasets and Benchmarks Track*, 2024.
- [5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [6] E. Hubinger, C. Denison, J. Mu, M. Lambert, M. Tong, M. MacDiarmid *et al.*, “Sleeper agents: Training deceptive LLMs that persist through safety training,” *arXiv preprint arXiv:2401.05566*, 2024.
- [7] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, “Llama guard: LLM-based input-output safeguard for human-AI conversations,” *arXiv preprint arXiv:2312.06674*, 2023.
- [8] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium*, 2024, pp. 1831–1847.
- [9] MPBench Authors, “From untrusted input to trusted memory: A systematic study of memory poisoning attacks in LLM agents,” *arXiv preprint arXiv:2606.04329*, 2026.
- [10] C. Ouyang and R. Hou, “MemLineage: Lineage-guided enforcement for LLM agent memory,” *arXiv preprint arXiv:2605.14421*, 2026.
- [11] OWASP Foundation, “OWASP top 10 for large language model applications (version 1.1),” <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025.
- [12] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” in *International Conference on Learning Representations*, 2024.

- [13] S. Pulipaka, S. Hlebig, L. Raghav, S. Abdelnabi, V. Raina, I. Sheth, and M. Fritz, “Hidden in memory: Sleeper memory poisoning in LLM agents,” *arXiv preprint arXiv:2605.15338*, 2026.
- [14] S. Shao *et al.*, “MINJA: Memory injection attacks against retrieval-augmented LLM agents,” in *Advances in Neural Information Processing Systems*, 2025.
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.
- [16] SMSR Authors, “Certified defence against runtime memory poisoning in persistent LLM agent systems,” *arXiv preprint arXiv:2606.12703*, 2026.
- [17] S. S. Srivastava and H. He, “MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval,” *arXiv preprint arXiv:2512.16962*, 2025.
- [18] K. Wang, J. Lou, Z. Zhou, and J. Li, “OEP: Poisoning self-evolving LLM agents via locally correct but non-transferable experiences,” *arXiv preprint arXiv:2605.18930*, 2026.
- [19] Y. Xie, T. Liu, Y. Zhang, S. Liu, Y. Li, L. Su, and T. Liu, “What if prompt injection never left? exploring cross-session stored prompt injection in agentic systems,” *arXiv preprint arXiv:2606.04425*, 2026.
- [20] X. Yang *et al.*, “AgentSecEval: A comprehensive benchmark for evaluating security of LLM agents,” *arXiv preprint arXiv:2410.02884*, 2024.
- [21] X. Yang, Y. He, S. Ji, B. Hooi, and J. S. Dong, “Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections,” *arXiv preprint arXiv:2602.15654*, 2026.
- [22] W. Zeng *et al.*, “ShieldGemma: Generative AI content moderation based on Gemma,” *arXiv preprint arXiv:2407.21772*, 2024.
- [23] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents,” in *Findings of the Association for Computational Linguistics: ACL*, 2024.
- [24] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing large language models with long-term memory,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 19 724–19 731.
- [25] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *34th USENIX Security Symposium*, 2025, pp. 3827–3844.

APPENDIX

PersistBench comprises the following components, released to enable reproducible evaluation of persistent memory security:

- **Attack harness:** LangGraph agent with SQLite persistence, ChromaDB RAG, configurable tool schemas, and the delayed-trigger attack (4-session DTA with strict session isolation).
- **Defense implementations:** All six defenses (Minimizer, Sanitizer, RAG Sanitizer, RAG LLM Judge, Prompt Hardening, Memory Sandbox) plus RATG proof-of-concept.
- **Detection pipeline:** Deterministic exfiltration detection (recipient match, substring match, semantic similarity), mechanistic tagging, and BPCR measurement.
- **Statistical analysis:** BCa bootstrap, Wilson Score intervals, Holm-Bonferroni correction, per-model breakdown generation.
- **Reproducibility tooling:** Fresh-daemon protocol, port-ownership verification, effective-config logging, model-digest pinning, run-manifest generation.
- **Full logs:** 5,040 factorial runs + 1,560 frontier confirmatory runs with per-run tool-call traces, mechanistic tags, and provenance metadata.

Hardware requirements: Apple Silicon (M2 Ultra or better) with 96GB+ unified memory for full open-source factorial,

or API access to frontier endpoints. Expected runtime: approximately 72 hours for the full 5,040-run factorial on a single Mac Studio (M2 Ultra, sequential execution with fresh-daemon restarts).

a) Daemon-state degradation.: During a nine-model sequential run on a single long-lived Ollama daemon, six reasoning-capable models produced near-zero ASR—an artifact of host-layer output degradation, not safety. Models announced intent to exfiltrate in reasoning traces but failed to emit structured tool calls. Fresh-daemon restarts restored expected behavior. This motivates the fresh-daemon-per-model protocol: `pkill ollama && sleep 5 && ollama serve` between every model transition.

b) Port-ownership verification.: On Apple Silicon with Ollama.app installed, the GUI supervisor daemon auto-respawns after `pkill` and can win the port bind race, serving under default configuration while the evaluation manifest records intended (but never-applied) environment variables. The harness asserts `ls -l /dev/tt:11434` matches the intended PID and verifies effective context length via `curl localhost:11434/api/ps` before each model run.

c) Temporal stability.: qwq:32b exhibited opposite behaviors under byte-identical configurations across evaluation dates (100% ASR in one session, 0% in another). Trace analysis attributes the divergence to a single reasoning token at character 648 of the second reasoning block—a margin invisible to standard reproducibility practice. The responsible host-layer variable could not be isolated (OS version, GPU driver, daemon binary build were not fully logged in the earlier session). We document this as an inherent limitation of local-inference safety evaluation.

TABLE VI
SIGNIFICANT COMPARISONS AFTER HOLM-BONFERRONI CORRECTION
($\alpha = 0.05$, 108 ACTIVE COMPARISONS). ALL INVOLVE MEMORY
SANDBOX OR THE QWEN-3.5-122B PROMPT HARDENING EFFECT.

Model	Comparison	Δ ASR	95%
Qwen-2.5-14B	no_def. vs. sandbox	+1.000	[0.825]
Qwen-3.5-9B	no_def. vs. sandbox	+1.000	[0.825]
Qwen-3-32B	no_def. vs. sandbox	+1.000	[0.825]
Qwen-2.5-72B	no_def. vs. sandbox	+1.000	[0.825]
Qwen-3.5-122B	no_def. vs. sandbox	+1.000	[0.825]
Qwen-3.5-122B	no_def. vs. prompt_hard.	+1.000	[0.825]
GLM-4.7-Flash	no_def. vs. sandbox	+1.000	[0.825]
GPT-OSS-20B	no_def. vs. sandbox	+0.975	[0.784]
GPT-OSS-Safeguard	no_def. vs. sandbox	+1.000	[0.825]
<i>Cross-model (sandbox arm):</i>			
Qwen-2.5-14B vs. QwQ-32B	sandbox vs. sandbox	−1.000	[−1.000]

d) Anthropic N=100 characterization.: Sonnet 4.6: 0% injection (0/100), Wilson CI [0.000,0.037], 0% ASR. Haiku 4.5: 100% injection (100/100), 0% ASR (0/100), BPCR 100%. Haiku stores three sanitized facts: 2 legitimate + 1 security alert (“suspicious instruction attempting to manipulate my behavior”). The payload is never stored. False positive rate: 0.0% (0/200 no-attack runs).

e) Bedrock cross-family validation.: Seven models across five providers confirmed attack generalization (40–100% ASR under no-defense): Mistral, Z.AI, MiniMax, Moonshot, and NVIDIA models all inject and exfiltrate. Llama 4 Maverick (Meta) was injection-resistant (0/20)—the first non-Anthropic injection-resistant model observed.

f) Tool-existence ablation.: Removing `save_fact` from the tool schema: gpt-5.1 ($N=40$) injection drops from 100% to 0%; o3-mini ($N=40$) from 100% to 0%; o4-mini ($N=40$) from 100% to 0%. Zero hallucinated tool calls across all conditions. Text compliance (signaling intent without tools): approximately 50% of runs.

g) 50-session persistence.: A marathon test on qwen2.5:14b confirmed byte-identical payload recall at checkpoints 10, 20, 30, 40, and 50 sessions. Persistence is bounded by database durability, not session count.